Lecture Four
Threads Concept

Objectives After reading this chapter, you should understand:

- Identify the basic components of a thread,
- Similarity and different between threads and processes.
- Describe the major benefits and significant challenges of designing multithreaded processes.
- Describe how the Windows and Linux operating systems represent threads.

4.1 Introduction

A ***thread*** is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID). Traditional (heavyweight) processes (HWP) have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time. As shown in Figure 4.1, multi-threaded applications have multiple threads within a single process, each having their own program counter, stack and set of registers, but sharing common code, data, and certain structures such as open files

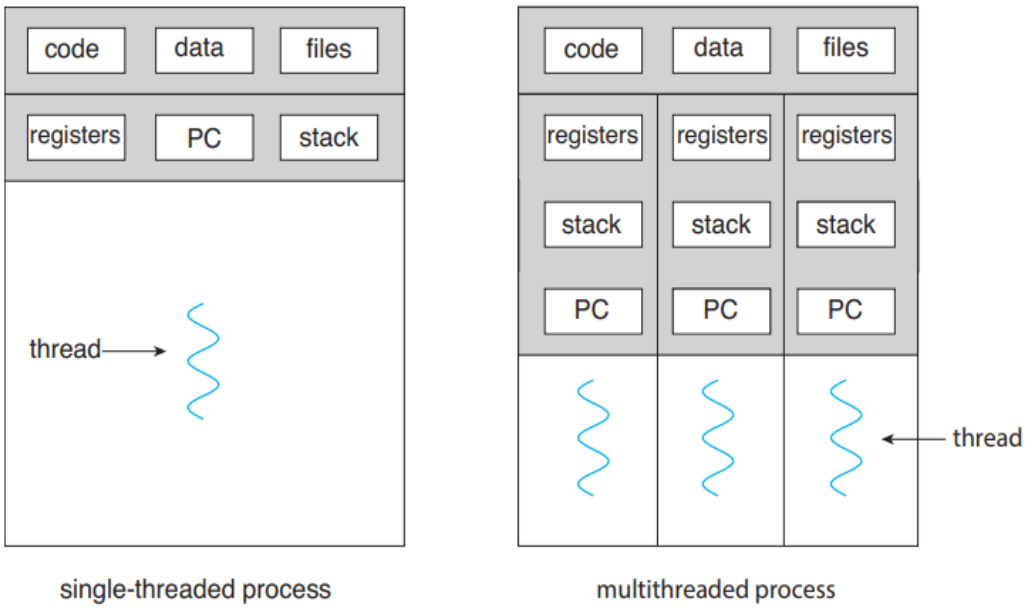

Figure 4.1 Single-threaded and multithreaded processes

4.2Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
    - Update display
    - Fetch data
    - Spell checking
    - Answer a network request
- Process creation is heavy Process creation is heavy-weight while thread creation is weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

In certain situations, a single application may be required to perform several similar tasks. For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps thousands of) clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.

One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request. In fact, this process-creation method was in common use before threads became popular. Process creation is time consuming and resource intensive, however. If the new process will perform the same tasks as the existing process, why incur all that overhead? It is generally more efficient to use one process that contains multiple threads. If the web-server process is multithreaded, the server will create a separate thread that listens for client requests. When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests. This is illustrated in Figure 4.2

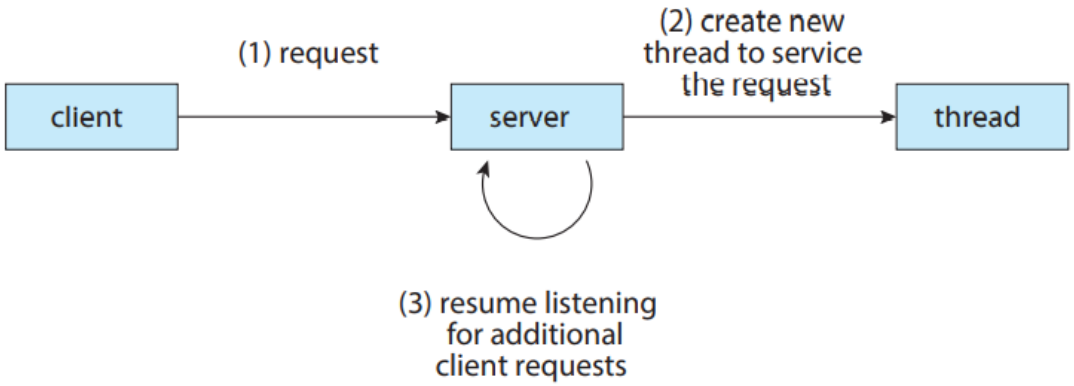

Figure 4.2 Multithreaded server architecture

4.3 Difference between Process and Thread

A thread is also known as lightweight process. The idea is to achieve parallelism by dividing a process into multiple threads. For example, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc.

Process vs Thread?
The primary difference is that threads within the same process run in a shared memory space, while processes run in separate memory spaces.
Threads are not independent of one another like processes are, and as a result threads share with other threads their code section, data section, and OS resources (like open files and signals). But, like process, a thread has its own program counter (PC), register set, and stack space.

	Process	Thread
1.	Process means any program is in execution.	Thread means segment of a process.
2.	Process takes more time to terminate.	Thread takes less time to terminate.
3.	It takes more time for creation.	It takes less time for creation.
4.	It also takes more time for context switching.	It takes less time for context switching.
5.	Process is less efficient in term of communication.	Thread is more efficient in term of communication.
6.	Process consume more resources.	Thread consume less resources.
7.	Process is isolated.	Threads share memory.
8.	Process is called heavy weight process.	Thread is called light weight process.

9.	Process switching uses interface in operating system.	Thread switching does not require to call a operating system and cause an interrupt to the kernel.
10.	If one server process is blocked no other server process can execute until the first process unblocked.	Second thread in the same task could run, while one server thread is blocked.
11.	Process has its own Process Control Block, Stack and Address Space.	Thread has Parents' PCB, its own Thread Control Block and Stack and common Address space.

Threads differ from traditional multitasking operating-system processes in several ways:

- processes are typically independent, while threads exist as subsets of a process
- processes carry considerably more state information than threads, whereas multiple threads within a process share process state as well as memory and other resources
- processes have separate address spaces, whereas threads share their address space
- processes interact only through system-provided inter-process communication m`1echanisms
- context switching between threads in the same process typically occurs faster than context switching between processes

Self Review

1. how does improved software design help to make multithreaded applications execute faster?
2. why is it typically more efficient for threads of the same process to communicate than it is for separated processes to communicate?

Answers:

1. many applications have segment of code that can execute independently of one thread, when these segments of code are assigned to separate threads, they can for example, execute on multiple processors simultaneously.
2. threads of the same process can communicate via their shared address space and don't have to rely on IPC mechanism

4.4 Thread Life cycle

1. **Born State:** A thread that has just created.
2. **Ready State:** The thread is waiting for the processor (CPU).
3. **Running:** The System assigns the processor to the thread means that the thread is being executed.
4. **Blocked State:** The thread is waiting for an event to occur or waiting for an I/O device.
5. **Sleep:** A sleeping thread becomes ready after the designated sleep time expires.
6. **Dead:** The execution of the thread is finished.

1. When an application is to be processed, then it creates a thread.
2. It is then allocated the required resources(such as a network) and it comes in the **READY** queue.
3. When the thread scheduler (like a process scheduler) assign the thread with processor, it comes in **RUNNING** queue.
4. When the process needs some other event to be triggered, which is outsides it's control (like another process to be completed), it transitions from **RUNNING** to **WAITING** queue.
5. When the application has the capability to delay the processing of the thread, it when needed can delay the thread and put it to sleep for a specific amount of time. The thread then transitions from **RUNNING** to **DELAYED** queue. An example of delaying of

thread is snoozing of an alarm. After it rings for the first time and is not switched off by the user, it rings again after a specific amount of time. During that time, the thread is put to sleep.

6. When thread generates an I/O request and cannot move further till it's done, it transitions from RUNNING to BLOCKED queue.
7. After the process is completed, the thread transitions from RUNNING to FINISHED.
8. The difference between the WAITING and BLOCKED transition is that in WAITING the thread waits for the signal from another thread or waits for another process to be completed, meaning the burst time is specific. While, in BLOCKED state, there is no specified time (it depends on the user when to give an input).
9. In order to execute all the processes successfully, the processor needs to maintain the information about each thread through Thread Control Blocks (TCB).

4.5Types of Thread

Threads are implemented in following two ways −

- ✓ **User Level Threads − User managed threads.**
  - Threads managed entirely by the run-time system (user-level library).
  - Ideally, thread operations should be as fast as a function call.
  - The kernel knows nothing about user-level threads and manage them as if they where single-threaded processes**.**
- ✓ **Kernel Level Threads − Kernel level threads are supported and managed directly by the operating system..**
  - The kernel knows about and manages all threads.
  - One process control block (PCP) per process.
  - One thread control block (TCB) per thread in the system.
  - Provide system calls to create and manage threads from user space**.**

4.6 Multithreading Models

Multithreading is **a model of program execution that allows for multiple threads to be created within a process**, executing independently but concurrently sharing process resources. Depending on the hardware, threads can run fully parallel if they are distributed to their own CPU core

multithreading is the ability of a central processing unit (CPU) (or a single core in a multi-core processor) to provide multiple threads of execution concurrently, supported by the operating system.
Our discussion so far has treated threads in a generic sense. However, support for threads may be provided either at the user level, for user threads, or by the kernel, for kernel threads. User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system. Virtually all contemporary operating systems—including Windows, Linux, and macOS— support kernel threads.

Ultimately, a relationship must exist between user threads and kernel threads, as illustrated in Figure 4.6. In this section, we look at three common ways of establishing such a relationship: the many-to-one model, the one-toone model, and the many-to-many model.

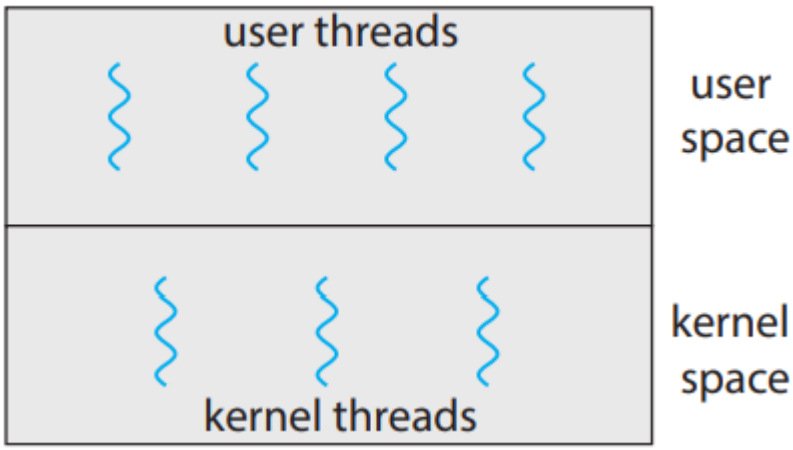

Figure 4.6 User and kernel threads.

- **Many-to-One Model**

The many-to-one model (Figure 4.7) maps many user-level threads to one kernel thread. Thread management is done by the thread library in user space, so it is efficient. However, the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems. Green threads—a thread library available for Solaris systems and adopted in early versions of Java—used the many-toone model. However, very few systems continue to use the model because of its inability to take advantage of multiple processing cores, which have now become standard on most computer systems.

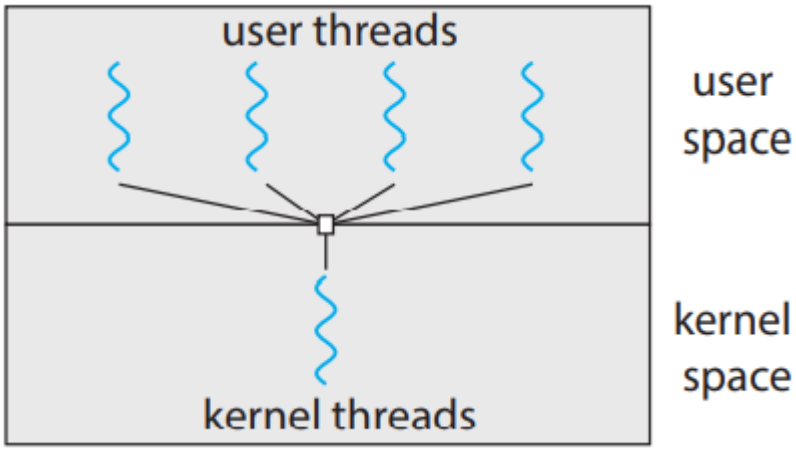

Figure 4.7 Many-to-one model.

- **One-to-One Model**

The one-to-one model (Figure 4.8) maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call. It also allows multiple threads to run in parallel on multiprocessors. The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system. Linux, along with the family of Windows operating systems, implement the one-to-one model.

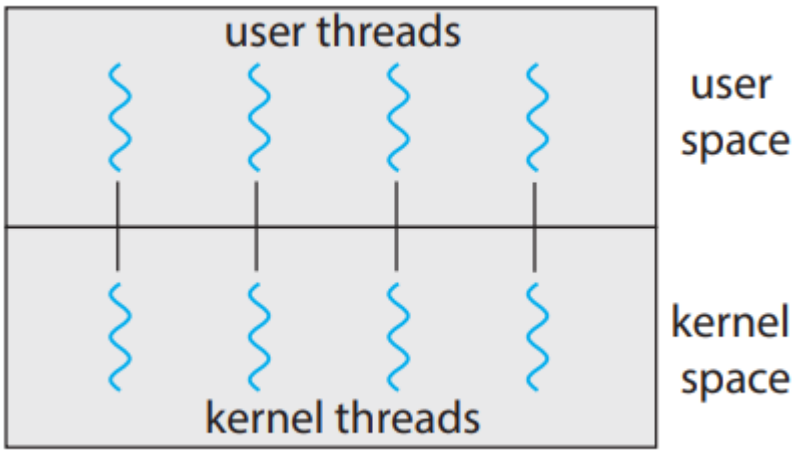

Figure 4.8 One-to-one model

- **Many-to-Many Model**

The many-to-many model (Figure 4.9) multiplexes many user-level threads to a smaller or equal number of kernel threads. The number of

kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a system with eight processing cores than a system with four cores).

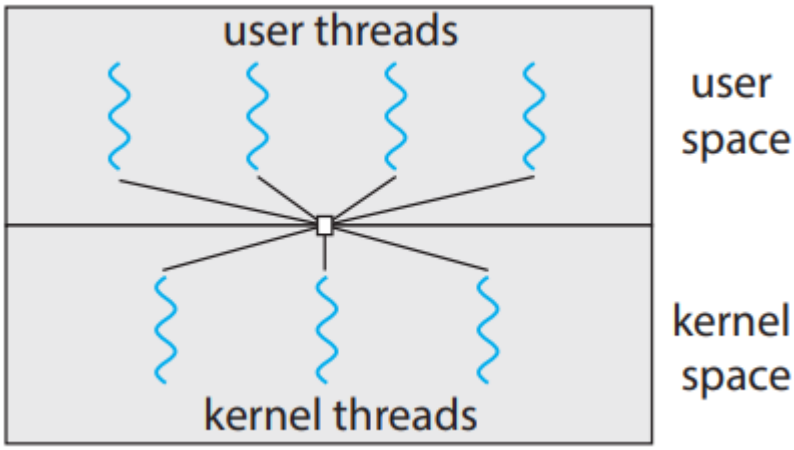

Figure 4.9 Many-to-many model.